

Prompts

ahora el profe me dejo crear otra clase

me pide esto

Contexto:

Ya tengo un proyecto desarrollado en clase con Python y programacion orientada a objetos.
En el

proyecto ya existe una clase Stack o Pila y dos aplicaciones que utilizan esta estructura.
Ahora

necesito agregar una nueva estructura de datos: Cola o Queue.

Objetivo:

Necesito crear una clase Queue con las operaciones basicas enqueue, dequeue, printQueue, firstQueue

y lastQueue. La clase debe integrarse al mismo proyecto existente sin modificar lo que ya funciona.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces ni modifiques la clase Stack existente.
3. No elimines las aplicaciones ya desarrolladas con pila.
4. Respeta la organizacion actual de archivos, clases y nombres del proyecto.
5. La solucion debe estar hecha con programacion orientada a objetos.
6. Propón cambios minimos e indica exactamente en que archivo y en que parte deben colocarse.
7. La clase Queue debe tener al menos los metodos enqueue, dequeue, printQueue, firstQueue y lastQueue.
8. Si se requiere un metodo auxiliar para saber si la cola esta vacia, proponlo sin afectar la lista de operaciones solicitadas.
9. Explica brevemente como funciona FIFO y como se diferencia de una pila.
10. Proporciona pruebas para comprobar que la clase funciona correctamente.

Codigo existente:

[Pega aqui la clase Stack]

[Pega aqui la clase Queue sugerida]

[Pega aqui la estructura de archivos del proyecto]

[Pega aqui el codigo de main.py]

[Pega aqui la clase que carga la interfaz o menu, si aplica]

Resultado esperado de tu respuesta:

- Primero analiza la estructura del codigo existente.
- Despues propone la clase Queue.
- Luego indica como integrarla al proyecto sin afectar Stack ni las aplicaciones existentes.
- Finalmente proporciona una prueba guiada de enqueue, dequeue, printQueue, firstQueue y lastQueue.
- No omitas explicacion, porque debo entender el codigo para defenderlo en clase.

Resultado

¡Perfecto! Vamos a diseñar la estructura de la Cola (Queue) respetando minuciosamente cada una de las condiciones impuestas por tu profesor y manteniendo la coherencia de tu proyecto, usando nodos en inglés (`data` , `next`) tal como hiciste con la Pila.

1. Análisis de la estructura y analogía (Pila vs. Cola)

Antes de programar, necesitas entender la diferencia clave para defender tu proyecto en clase:

- **Pila (Stack) - LIFO (*Last In, First Out*):** El último en llegar es el primero en salir (como una pila de platos). Solo tienes acceso a un extremo: el `top`.
- **Cola (Queue) - FIFO (*First In, First Out*):** El primero en llegar es el primero en ser atendido (como la fila del banco). Aquí se interactúa con **ambos extremos**:
 - Los nuevos elementos entran por el final (`back` o `last`).
 - Los elementos que salen se retiran por el frente (`front` o `first`).

+ Pregunta a Gemini

Flash ▾ 

```
from estructuras.lineales.nodo import Node
```

```
class LinkedQueue:

    def __init__(self):

        # Una cola necesita apuntar al inicio (front) para sacar elementos

        # y al final (back) para insertar elementos rápidamente.

        self.front = None

        self.back = None


    def is_empty(self):

        """Método auxiliar solicitado para comprobar si la cola está vacía."""

        return self.front is None


    def enqueue(self, dato):

        """Inserta un elemento al final de la cola (Back)."""

        nuevo_nodo = Node(dato)

        # Si la cola está vacía, el nuevo nodo es tanto el frente como el final

        if self.is_empty():

            self.front = nuevo_nodo

            self.back = nuevo_nodo

        else:

            # El nodo que estaba al final ahora apunta al nuevo nodo

            self.back.next = nuevo_nodo

            # Actualizamos nuestro puntero del final hacia el nuevo nodo

            self.back = nuevo_nodo


    def dequeue(self):

        """Elimina el elemento al frente de la cola (Front). Retorna False si está vacía."""

        if self.is_empty():
```

```
return False
```

```
# Avanzamos el frente al siguiente nodo de la fila
```

```
self.front = self.front.next
```

```
# Si al avanzar el frente quedó vacío, significa que la cola se quedó sin elementos
```

```
if self.front is None:
```

```
    self.back = None
```

```
return True
```

```
def firstQueue(self):
```

```
    """Retorna el dato del primer elemento en la cola sin eliminarlo."""
```

```
    if self.is_empty():
```

```
        return None
```

```
    return self.front.data
```

```
def lastQueue(self):
```

```
    """Retorna el dato del último elemento en la cola sin eliminarlo."""
```

```
    if self.is_empty():
```

```
        return None
```

```
    return self.back.data
```

```
def printQueue(self):
```

```
    """Recorre la cola y devuelve una cadena de texto representativa."""
```

```
    if self.is_empty():
```

```
        return "Cola vacía"
```

```
elementos = []
```

```

actual = self.front

while actual:

    elementos.append(str(actual.data))

    actual = actual.next

# Representación visual simulando una fila india (Frente <- ... <- Final)
return " <- ".join(elementos)

```

```

from PyQt5.QtWidgets import QDialog, QMessageBox
from PyQt5.uic import loadUi
from estructuras.lineales.lista_enlazadaCola import LinkedQueue

```

```

class MenuCola(QDialog):

    def __init__(self):
        super().__init__()
        loadUi("ui/cola.ui", self)

        self.cola = LinkedQueue()

        # Conexiones respetando las operaciones del profesor
        self.btn_enqueue.clicked.connect(self.ejecutar_enqueue)
        self.btn_dequeue.clicked.connect(self.ejecutar_dequeue)
        self.btn_first.clicked.connect(self.ver_primer)
        self.btn_last.clicked.connect(self.ver_ultimo)
        self.btn_print.clicked.connect(self.mostrar_en_ventana)

    def ejecutar_enqueue(self):
        dato = self.txt_dato.text().strip()

```

```
if dato == "":
    QMessageBox.warning(self, "Error", "Ingrese un dato válido")
    return
self.cola.enqueue(dato)
self.txt_dato.clear()
self.actualizar_interfaz()

def ejecutar_dequeue(self):
    if not self.cola.dequeue():
        QMessageBox.warning(self, "Error", "La cola está vacía")
        return
    self.actualizar_interfaz()

def ver_primero(self):
    primero = self.cola.firstQueue()
    if primero is None:
        QMessageBox.warning(self, "First", "La cola está vacía")
    else:
        QMessageBox.information(self, "Primer Elemento", f"El frente de la cola es: {primero}")

def ver_ultimo(self):
    ultimo = self.cola.lastQueue()
    if ultimo is None:
        QMessageBox.warning(self, "Last", "La cola está vacía")
    else:
        QMessageBox.information(self, "Último Elemento", f"El final de la cola es: {ultimo}")

def mostrar_en_ventana(self):
```

```
QMessageBox.information(self, "Print Queue", f"Estado de la cola:\n\n{self.cola.printQueue()}")
```

```
def actualizar_interfaz(self):  
    self.lbl_resultado.setText(self.cola.printQueue())
```

interferir con las otras:

1. Al principio del archivo (Imports):

Python

```
from load.load_lista_cola import MenuCola
```

2. En el constructor `__init__` (Conexión de la acción): (Suponiendo que agregaste el botón o menú llamado `actionColas` en tu ventana principal)

Python

```
self.actionColas.triggered.connect(self.abrir_colas)
```

+ Pregunta a Gemini

Flash



ahora dame un estilo css para el qt

¡Listo! Para la ventana de la Cola (Queue) vamos a utilizar un esquema de colores basado en tonos verdes (Teal/Esmeralda). El verde se asocia psicológicamente con los sistemas de flujo, atención y procesamiento ordenado (como "pase", "siguiente" o "libre"), lo que encaja perfecto con la lógica de una fila de banco o atención al cliente (FIFO).

Copia y pega este código QSS en la propiedad `styleSheet` del `Dialog` de tu nueva ventana de Colas en Qt Designer: